# C++ Primer 习题集

（第5版）

Stanley B. Lippman
[美] Josée Lajoie 著
Barbara E. Moo

王刚 杨巨峰 李忠伟 改编

電子工業出版社
Publishing House of Electronics Industry
北京•BEIJING

## 内 容 简 介

*C++ Primer（Fifth Edition）*由三位 C++大师 Stanley B. Lippman、Josée Lajoie 和 Barbara E. Moo 合作完成，其中文译本《C++ Primer 中文版（第 5 版）》也已出版。与上一版相比，这一版基于 C++11 标准进行了全面而彻底的内容更新；既是初学者的最佳学习指南，也是中高级程序员不可或缺的参考书。本书作为这部久负盛名的 C++经典教程的配套习题解答，提供了 *C++ Primer（Fifth Edition）*中英文版图书中所有习题的参考答案。

本书对使用 *C++ Primer（Fifth Edition）*学习 C++程序设计语言的读者来说是一本非常理想的参考书。

Authorized translation from the English language edition, entitled C++ Primer, 5E, 9780321714114 by STANLEY B. LIPPMAN; JOSEE LAJOIE; BARBARA E. MOO, published by Pearson Education, Inc., Copyright©2013 Pearson Education, Inc.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education,Inc.

Chinese simplified language edition published by PEARSON EDUCATION ASIA LTD., and PUBLISHING HOUSE OF ELECTRONICS INDUSTRY Copyright © 2015.

本书翻译改编专有出版权由 Pearson Education 培生教育出版亚洲有限公司授予电子工业出版社。未经出版者预先书面许可，不得以任何方式复制或抄袭本书的任何部分。

本书翻译改编版贴有 Pearson Education 培生教育出版集团激光防伪标签，无标签者不得销售。

版权贸易合同登记号 图字：01-2013-5743

**图书在版编目（CIP）数据**

C++ Primer 习题集：第 5 版/（美）李普曼（Lippman,S.B.），（美）拉乔伊（Lajoie,J.），（美）默（Moo,B.E.）著；王刚，杨巨峰，李忠伟改编. —北京：电子工业出版社，2015.3

书名原文：C++ Primer, 5E

ISBN 978-7-121-25229-7

Ⅰ. ①C… Ⅱ. ①李… ②拉… ③默… ④王… ⑤杨… ⑥李… Ⅲ. ①C 语言－程序设计－习题集 Ⅳ.①TP312-44

中国版本图书馆 CIP 数据核字（2014）第 302751 号

策划编辑：张春雨
责任编辑：刘 舫
印　　刷：北京京科印刷有限公司
装　　订：三河市皇庄路通装订厂
出版发行：电子工业出版社
　　北京市海淀区万寿路 173 信箱　　邮编：100036
开　　本：787×1092　1/16　　印张：33　　字数：740 千字
版　　次：2015 年 3 月第 1 版
印　　次：2015 年 4 月第 2 次印刷
定　　价：89.00 元

凡所购买电子工业出版社图书有缺损问题，请向购买书店调换。若书店售缺，请与本社发行部联系，联系及邮购电话：（010）88254888。

质量投诉请发邮件至 zlts@phei.com.cn，盗版侵权举报请发邮件至 dbqq@phei.com.cn。

服务热线：（010）88258888。

# 前言

C++语言是应用极为广泛的一门程序设计语言，难以计数的程序员已经通过旧版的 *C++ Primer* 学会了 C++语言。

2011 年，C++标准委员会发布了 ISO C++标准的一个重要修订版。此修订版是 C++进化过程中的最新一步，其目标是使得 C++语言更统一、更简单、更安全、更高效。为此，三位 C++大师 Stanley B. Lippman、Josée Lajoie 和 Barbara E. Moo 编著完成了 *C++ Primer（Fifth Edition）*，在旧版基础上基于 C++11 标准进行了全面而彻底的内容更新，重点讲解了 C++11 新特性以及这些新特性是如何影响 C++语言的。*C++ Primer（Fifth Edition）*的中文译本《C++ Primer 中文版（第 5 版）》也已由电子工业出版社出版。完整代码：www.wzbook.org

本书是 *C++ Primer(Fifth Edition)*的配套书籍，提供了该书所有习题的参考答案。在设计本书的行文格式时我们做了如下考虑：

- 在原书和习题解答之间建立良好衔接。
- 授之以鱼，不如授之以渔，要让读者明白怎么做以及为什么。
- 与 C++11 的目标保持一致，即简单高效。

在每章的开始，我们设计了“导读”模块，目的是言简意赅地把本章的重要知识点串成一个整体，帮助读者梳理在本章应该学到哪些内容。之后的每个题目分为【出题思路】和【解答】两部分：其中【出题思路】站在出题者的角度阐述题目的考查角度和考查目的，【解答】则负责给出满足题目要求的代码及注释；为了开拓读者的思路，有的题目我们给出了不止一种解决方案。

衷心希望本书能对使用 *C++ Primer（Fifth Edition）*学习 C++的读者有所帮助。

由于编者水平有限，书中的不当之处恳请读者批评指正。

编　者

2014 年 11 月

于南开园

# 目录

第 1 章　开始 ..... 1

练习 1.1～练习 1.25

第 2 章　变量和基本类型 ..... 12

练习 2.1～练习 2.42

第 3 章　字符串、向量和数组 ..... 37

练习 3.1～练习 3.45

第 4 章　表达式 ..... 80

练习 4.1～练习 4.38

第 5 章　语句 ..... 99

练习 5.1～练习 5.25

第 6 章　函数 ..... 120

练习 6.1～练习 6.56

第 7 章　类 ..... 152

练习 7.1～练习 7.58

第 8 章　IO 库 ..... 183

练习 8.1～练习 8.14

第 9 章　顺序容器 ..... 193

练习 9.1～练习 9.52

第 10 章　泛型算法 ..... 234

练习 10.1～练习 10.42

第 11 章 关联容器 ..... 273

练习 11.1～练习 11.38

第 12 章 动态内存 ..... 297

练习 12.1～练习 12.33

第 13 章 拷贝控制 ..... 331

练习 13.1～练习 13.58

第 14 章 重载运算与类型转换 ..... 368

练习 14.1～练习 14.53

第 15 章 面向对象程序设计 ..... 399

练习 15.1～练习 15.42

第 16 章 模板与泛型编程 ..... 424

练习 16.1～练习 16.67

第 17 章 标准库特殊设施 ..... 458

练习 17.1～练习 17.39

第 18 章 用于大型程序的工具 ..... 483

练习 18.1～练习 18.30

第 19 章 特殊工具与技术 ..... 502

练习 19.1～练习 19.26

# 第 1 章
# 开始

导读

在本章，我们初识 C++语言，学习了如何编译、运行简单的 C++程序。因此，本章习题的一个主要训练内容是熟悉编译工具和集成开发环境。本章还介绍了 C++程序的基本结构，如何定义变量，如何进行输入输出，以及如何编写 if、for 和 while 语句，用这些知识来练习编写简单的 C++程序是本章练习的另一个重要内容。本章最后简要介绍了如何使用其他人定义的类，因此，本章最后一部分习题是利用书中的 Sales-item 类进行类对象的创建、使用方面的练习。

**练习 1.1：**查阅你使用的编译器的文档，确定它所使用的命名约定。编译并运行第 2 页的 main 程序。

【出题思路】

熟悉编译工具和集成开发环境。

【解答】

首先利用编辑器（如 Linux 系统中的 vim 或 Windows 系统中 Visual Studio 自带的编辑器）输入 main 程序，保存为.cpp 或.cc 后缀的源程序文件。然后按书中说明运行 GNU 或微软编译器，将源文件编译为可执行文件。最后执行程序，观察执行结果。

尝试使用命令行方式编译程序，尝试不同编译选项，可以帮助我们更好地了解编译过程，掌握生成所需目标程序的方法。

**练习 1.2**：改写程序，让它返回-1。返回值-1 通常被当作程序错误的标识。重新编译并运行你的程序，观察你的系统如何处理 main 返回的错误标识。

【出题思路】

了解 C++程序与操作系统间的交互。

【解答】

Windows 7 操作系统并不处理或报告程序返回的错误标识，直观上，返回-1 的程序与返回 0 的程序在执行效果上并无不同。但环境变量 ERRORLEVEL 记录了上一个程序的返回值。因此，在控制台窗口执行修改后的程序，接着执行 echo %ERRORLEVEL%，会输出-1。在 Linux 系统中，执行 echo $?有类似效果。

**练习 1.3**：编写程序，在标准输出上打印 Hello, World。

【出题思路】

C++程序基本结构和简单输出的练习。

【解答】

```
#include<iostream>
int main()
{
  std::cout << "Hello, World" << std::endl;
  return 0;
}
```

**练习 1.4**：我们的程序使用加法运算符+来将两个数相加。编写程序使用乘法运算符*，来打印两个数的积。

【出题思路】

简单运算和简单输入输出的练习。

【解答】

```
#include <iostream>

int main()
{
  std::cout << "请输入两个数" << std::endl;
  int v1, v2;
  std::cin >> v1 >> v2;
  std::cout << v1 << "和" << v2 << "的积为"
    << v1 * v2 << std::endl;
  return 0;
}
```

**练习 1.5**：我们将所有输出操作放在一条很长的语句中。重写程序，将每个运算对象的打印操作放在一条独立的语句中。

【出题思路】

对输出语句进行不同形式的练习，同时让读者体会良好的程序格式。

【解答】

```
#include <iostream>

int main()
{
  std::cout << "请输入两个数";
  std::cout << std::endl;
  int v1, v2;
  std::cin >> v1 >> v2;
  std::cout << v1 << "和" << v2 << "的积为"
    << v1 * v2 << std::endl;
  return 0;
}
```

**练习 1.6：**解释下面程序片段是否合法。

```
std::cout << "The sum of " << v1;
          << " and " << v2;
          << " is " << v1 + v2 << std::endl;
```

如果程序是合法的，它输出什么？如果程序不合法，原因何在？应该如何修正？

【出题思路】

延续练习 1.5，让读者体会输出语句分段形式容易出现的错误。

【解答】

这段代码不合法。

前两行的末尾有分号，表示语句结束，第 2、3 两行为两条新的语句。而这两条语句在“<<”之前缺少了输出流，应在“<<”之前加上“std::cout”，即得到正确的程序。

**练习 1.7：**编译一个包含不正确的嵌套注释的程序，观察编译器返回的错误信息。

【出题思路】

一方面了解非法嵌套注释，另一方面体会编译器对较为复杂的错误如何给出错误信息，程序员应如何利用这些信息迅速定位、修改错误。

【解答】

对不正确的嵌套注释，不同编译器给出的错误信息可能是不同的，而且通常很难理解。例如，用 tdm-gcc 4.8.1 编译器编译 1.3 节中错误嵌套的程序：

```
#include <iostream>

/*
* 注释对/* */不能嵌套。
* “不能嵌套”几个字会被认为是源码，
* 像剩余程序一样处理
*/

int main()
```

```
{
  return 0;
}
```

编译器会报告：

```
4   error: stray '\262' in program
4   error: stray '\273' in program
```

（篇幅所限，后续错误信息略。）

原因是编译器将第一个“*/”看作注释结束，之后的中文文字看作下一条语句，从而给出非法字符的错误信息。如果“*/”之后是英文文字，或是使用其他编译器进行编译，给出的可能是完全不同的错误信息。而且这些错误信息都很难直接与注释错误嵌套挂上钩，程序员需要有一定的经验才能快速定位错误，确定错误原因。

**练习 1.8：**指出下列哪些输出语句是合法的（如果有的话）：

```
std::cout << "/*";
std::cout << "*/";
std::cout << /* "*/" */;
std::cout << /* "*/" /* "/*" */;
```

预测编译这些语句会产生什么样的结果，实际编译这些语句来验证你的答案（编写一个小程序，每次将上述一条语句作为其主体），改正每个编译错误。

【出题思路】

进一步熟悉更复杂的正确和不正确的注释语句。

【解答】

第一条和第二条语句显然是合法的。

在第三条语句中，第一个双引号被注释掉了，因此<<运算符后真正被编译的内容是" */，编译器认为这是一个不完整的字符串，所以会报告：

```
7   error: missing terminating " character
```

即，缺少结束的双引号。在分号前补上一个双引号，这条语句就变为正确的了。

第四条看起来很混乱，但它是正确的。第一个双引号被注释掉了，第四个双引号也被注释掉了，第二个双引号和第三个双引号之间的“ /* ”被认为是字符串的文字内容。但是，这样的程序风格显然是不好的。

**练习 1.9：**编写程序，使用 while 循环将 50 到 100 的整数相加。

【解答】

```
#include <iostream>

int main()
{
  int sum = 0;
  int i = 50;
  while (i <= 100) {
    sum += i;
    i++;
  }
```

```
  std::cout << "50 到 100 之间的整数之和为"
    << sum << std::endl;
  return 0;
}
```

**练习 1.10：**除了++运算符将运算对象的值增加 1 以外，还有一个递减运算符( -- )实现将值减少 1。编写程序，使用递减运算符在循环中按递减序打印出 10 到 0 之间的整数。

【出题思路】

递减循环较之递增循环用得较少，应有意进行这方面的练习，对提高编程能力是有益的。

【解答】

```
#include <iostream>

int main()
{
  int i = 10;
  while (i >= 0) {
    std::cout << i << " ";
    i--;
  }
  std::cout << std::endl;
  return 0;
}
```

**练习 1.11：**编写程序，提示用户输入两个整数，打印出这两个整数所指定的范围内的所有整数。

【出题思路】

编写一个简单但完整的依据用户输入进行处理的实例。让读者体会：用户的输入可能有各种各样的情况，我们编写的程序必须全面地考虑各种情况，避免由于考虑不周使得程序在某些用户输入下产生错误结果甚至更严重的后果。

【解答】

```
#include <iostream>

int main()
{
  std::cout << "请输入两个数";
  std::cout << std::endl;
  int v1, v2;
  std::cin >> v1 >> v2;
  if (v1 > v2)                    // 由大至小打印
    while (v1 >= v2) {
      std::cout << v1 << " ";
      v1--;
    }
```

```
  else                // 由小至大打印
    while (v1 <= v2) {
      std::cout << v1 << " ";
      v1++;
    }
  std::cout << std::endl;
  return 0;
}
```

**练习 1.12：**下面的 for 循环完成了什么功能？sum 的终值是多少？

```
int sum = 0;
for (int i = -100; i <= 100; ++i)
  sum += i;
```

【解答】

此循环将–100 到 100 之间（包含–100 和 100）的整数相加，sum 的终值是 0。

**练习 1.13：**使用 for 循环重做 1.4.1 节中的所有练习（第 11 页）。

【出题思路】

让读者体会：对同样的目标，C++语言提供的不同解决方法。

【解答】

练习 1.9 的循环版本：

```
#include <iostream>

int main()
{
  int sum = 0;
  for (int i = 50; i <= 100; i++)
    sum += i;
  std::cout << "50 到 100 之间的整数之和为"
    << sum << std::endl;
  return 0;
}
```

练习 1.10 的循环版本：

```
#include <iostream>

int main()
{
  for (int i = 10; i >= 0; i--)
    std::cout << i << " ";
  std::cout << std::endl;

  return 0;

}
```

练习 1.11 的循环版本：

```
#include <iostream>

int main()
```

```
{
  std::cout << "请输入两个数";
  std::cout << std::endl;
  int v1, v2;
  std::cin >> v1 >> v2;
  if (v1 > v2)  // 由大至小打印
    for (; v1 >= v2; v1--)
      std::cout << v1 << " ";
  else          // 由小至大打印
    for (; v1 <= v2; v1++)
      std::cout << v1 << " ";
  std::cout << std::endl;
  return 0;
}
```

**练习 1.14：**对比 for 循环和 while 循环，两种形式的优缺点各是什么？

【解答】

在循环次数已知的情况下，for 循环的形式显然更为简洁。

而循环次数无法预知时，用 while 循环实现更适合。用特定条件控制循环是否执行，循环体中执行的语句可能导致循环判定条件发生变化

**练习 1.15：**编写程序，包含第 14 页“再探编译”中讨论的常见错误。熟悉编译器生成的错误信息。

【出题思路】

继续熟悉编译器对不同错误给出的信息。

【解答】

对于复杂程序中的错误，编译器给出的错误信息很可能无法对应到真正的错误位置并给出准确的错误原因。这是很正常的，因为某些时候我们人类都无法准确判断程序员到底犯了什么错误，在当前人工智能技术发展水平下，要求编译器有超越人类的智能是不现实的。

而且，不同的编译器对同一个程序给出的错误信息有可能是有很大差别的。一方面是因为如前所述，很多时候并不存在“唯一正确”的错误原因，编译器（甚至我们人类也是）只能给出它认为最有可能的错误原因；另一方面，不同编译器对同样的错误原因也可能有自己不同的解释方式。

因此，使用几种不同的编译器，编译一些错误的程序，观察编译器给出的错误信息。对今后在大型软件中查找、修改编译错误是很有帮助的。

**练习 1.16：**编写程序，从 cin 读取一组数，输出其和。

【出题思路】

练习不定次数的循环，以及输入流结束判断。

【解答】

```
#include <iostream>
int main()
{
  int sum = 0, value = 0;
  std::cout << "请输入一些数，按 Ctrl+Z 表示结束"
    << std::endl;
  for (; std::cin >> value;)
    sum += value;
  std::cout << "读入的数之和为" <<
    sum << std::endl;
  return 0;
}
```

显然，对于循环次数无法预知的情况，for 循环比 while 循环稍累赘一些。

**练习 1.17：**如果输入的所有值都是相等的，本节的程序会输出什么？如果没有重复值，输出又会是怎样的？

【出题思路】

练习程序分析和手工执行程序。

【解答】

如果输入的所有值都相等，则 while 循环中的 else 分支永远不会执行，直到输入结束，while 循环退出，循环后的输出语句打印这唯一的一个值和它出现的次数。

若没有重复值，则 while 循环中的 if 语句的真值分支永远不会执行，每读入一个值，都会进入 else 分支，打印它的值和出现次数 1。输入结束后，while 循环退出，循环后的输出语句打印最后一个值和出现次数 1。

**练习 1.18：**编译并运行本节的程序，给它输入全都相等的值。再次运行程序，输入没有重复的值。

【解答】

输入：

```
1 1 1 1 1
```

程序输出：

```
1 occurs 5 times
```

输入：

```
1 2 3 4 5
```

程序输出：

```
1 occurs 1 times
2 occurs 1 times
3 occurs 1 times
4 occurs 1 times
5 occurs 1 times
```

注意，不要忘了用 Ctrl+Z 表示输入结束。

**练习 1.19**：修改你为 1.4.1 节练习 1.10（第 11 页）所编写的程序（打印一个范围内的数），使其能处理用户输入的第一个数比第二个数小的情况。

【解答】

练习 1.10 的解答已经包含了此功能。

**练习 1.20**：在网站 http://www.informit.com/title/0321714113 上，第 1 章的代码目录中包含了头文件 Sales_item.h。将它拷贝到你自己的工作目录中。用它编写一个程序，读取一组书籍销售记录，将每条记录打印到标准输出上。

【出题思路】

练习如何使用其他人定义的类来创建、使用对象。

【解答】

```
#include <iostream>
#include "Sales_item.h"
int main()
{
  Sales_item book;
  std::cout << "请输入销售记录："
    << std::endl;
  while (std::cin >> book) {
    std::cout << "ISBN、售出本数、销售额和平均售价为 "
      << book << std::endl;
  }
  return 0;
}
```

**练习 1.21**：编写程序，读取两个 ISBN 相同的 Sales_item 对象，输出它们的和。

【出题思路】

类对象的更复杂的使用，利用类接口进行运算。

【解答】

```
#include <iostream>
#include "Sales_item.h"
int main()
{
  Sales_item trans1, trans2;
  std::cout << "请输入两条 ISBN 相同的销售记录："
    << std::endl;
  std::cin >> trans1 >> trans2;
  if (compareIsbn(trans1, trans2))
    std::cout << "汇总信息：ISBN、售出本数、销售额和平均售价为 "
      << trans1 + trans2 << std::endl;
  else
```

```
    std::cout << "两条销售记录的 ISBN 不同" << std::endl;
  return 0;
}
```

**练习 1.22：**编写程序，读取多个具有相同 ISBN 的销售记录，输出所有记录的和。

【出题思路】

练习在处理数据流的过程中“状态”（是否是相同的 ISBN）的保存和变迁。

【解答】

```
#include <iostream>
#include "Sales_item.h"

int main()
{
  Sales_item total, trans;
  std::cout << "请输入几条 ISBN 相同的销售记录："
    << std::endl;
  if (std::cin >> total) {
    while (std::cin >> trans)
      if (compareIsbn(total, trans)) // ISBN 相同
        total = total + trans;
      else { // ISBN 不同
        std::cout << "ISBN 不同" << std::endl;
        return -1;
      }
    std::cout << "汇总信息：ISBN、售出本数、销售额和平均售价为 "
      << total << std::endl;
  }
  else {
    std::cout << "没有数据" << std::endl;
    return -1;
  }
  return 0;
}
```

**练习 1.23：**编写程序，读取多条销售记录，并统计每个 ISBN（每本书）有几条销售记录。

【解答】

```
#include <iostream>
#include "Sales_item.h"

int main()
{
  Sales_item trans1, trans2;
  int num = 1;
  std::cout << "请输入若干销售记录："
    << std::endl;
  if (std::cin >> trans1) {
    while (std::cin >> trans2)
```

```
      if (compareIsbn(trans1, trans2)) // ISBN 相同
        num++;
      else { // ISBN 不同
        std::cout << trans1.isbn() << "共有"
          << num << "条销售记录" << std::endl;
        trans1 = trans2;
        num = 1;
      }
    std::cout << trans1.isbn() << "共有"
      << num << "条销售记录" << std::endl;
  }
  else {
    std::cout << "没有数据" << std::endl;
    return -1;
  }
  return 0;
}
```

**练习 1.24：**输入表示多个 ISBN 的多条销售记录来测试上一个程序，每个 ISBN 的记录应该聚在一起。

【解答】

在网站 http://www.informit.com/title/0321714113 上，第 1 章的代码目录中包含了一些数据文件，可以将这些文件重定向到此程序进行测试，也可自己创建销售记录文件进行测试。

**练习 1.25：**借助网站上的 Sales_item.h 头文件，编译并运行本节给出的书店程序。

【出题思路】

练习编译、运行稍大规模的程序。

【解答】

在网站 http://www.informit.com/title/0321714113 上下载 Sales_item.h 头文件，编译书店程序即可。

# 第 2 章
# 变量和基本类型

导读

本章介绍了 C++的几种典型数据类型，它们分别是：

- 基本内置类型
- 复合类型
- 自定义数据结构

其中，char、int、long、float、double、bool 是最常见的基本内置类型；引用和指针是两种最重要的复合类型；struct 关键字和 class 关键字则常用于声明用户自定义的数据结构。

除此之外，本章的重要知识点还包括：变量的声明、定义和作用域，顶层 const 和底层 const。C++11 新标准引入了 auto 关键字和 decltype 关键字，利用这两个关键字，程序员就可以不必指定变量的数据类型，而把这项任务交给编译器自动执行。

**练习 2.1：**类型 int、long、long long 和 short 的区别是什么？无符号类型和带符号类型的区别是什么？float 和 double 的区别是什么？

【出题思路】

本题旨在考查C++语言中几种主要算术类型的区别以及符号的表示方法和意义，读者需要重点理解几种算术类型在内存中的存储方式。

【解答】

在 C++语言中，int、long、long long 和 short 都属于整型，区别是 C++标准规定的尺寸的最小值（即该类型在内存中所占的比特数）不同。其中，short 是短整型，占 16 位；int 是整型，占 16 位；long 和 long long 均为长整型，分

别占 32 位和 64 位。C++标准允许不同的编译器赋予这些类型更大的尺寸。某一类型占的比特数不同，它所能表示的数据范围也不一样。

大多数整型都可以划分为无符号类型和带符号类型，在无符号类型中所有比特都用来存储数值，但是仅能表示大于等于 0 的值；带符号类型则可以表示正数、负数或 0。

float 和 double 分别是单精度浮点数和双精度浮点数，区别主要是在内存中所占的比特数不同，以及默认规定的有效位数不同。

**练习 2.2：**计算按揭贷款时，对于利率、本金和付款分别应选择何种数据类型？说明你的理由。

【出题思路】

本题旨在考查 C++语言中选择数据类型的方法。

【解答】

在实际应用中，利率、本金和付款既有可能是整数，也有可能是普通的实数。因此应该选择一种浮点类型来表示。在三种可供选择的浮点类型 float、double 和 long double 中，double 和 float 的计算代价比较接近且表示范围更广，long double 的计算代价则相对较大，一般情况下没有选择的必要。综合以上分析，选择 double 是比较恰当的。

**练习 2.3：**读程序写结果。

```
unsigned u=10, u2=42;
std::cout << u2-u << std::endl;
std::cout << u-u2 << std::endl;

int i=10, i2=42;
std::cout << i2-i << std::endl;
std::cout << i-i2 << std::endl;
std::cout << i-u << std::endl;
std::cout << u-i << std::endl;
```

【出题思路】

本题考查的知识点包括：无符号数的计算、带符号数的计算以及混合计算。

【解答】

程序的运行结果是：

```
32
4294967264
32
-32
0
0
```

u 和 u2 都是无符号整数，因此 u2-u 得到了正确的结果（42–10=32）；u-u2 也能正确计算，但是因为直接计算的结果是-32，所以在表示为无符号整数时自动加上

了模，在作者的编译环境中 int 占 32 位，因此加模的结果是 4294967264。

i 和 i2 都是带符号整数，因此中间两个式子的结果比较直观，42-10=32，10-42=–32。

在最后两个式子中，u 和 i 分别是无符号整数和带符号整数，计算时编译器先把带符号数转换为无符号数，幸运的是，i 本身是一个正数，因此转换后不会出现异常情况，两个式子的计算结果都是 0。

不过需要提醒读者注意的是，一般情况下请不要在同一个表达式中混合使用无符号类型和带符号类型。因为计算前带符号类型会自动转换成无符号类型，当带符号类型取值为负时就会出现异常结果。

**练习 2.4：**编写程序检查你的估计是否正确，如果不正确，请仔细研读本节直到弄明白问题所在。

【出题思路】

本题旨在考查同时含有无符号类型和带符号类型的表达式的计算问题。

【解答】

对于读程序写结果的题目，读者首先应该根据本节知识点独立思考，在脑海中勾勒出题目的考查角度和关键点，进而写出结果并在编程环境中加以验证。

通过练习本题，尤其是观察 u-u2 的异常结果，进一步理解无符号类型和带符号类型的区别。读者不妨思考，你是否能用上述 4 个变量组合出其他产生异常结果的式子？应该如何避免这种情况？

**练习 2.5：**指出下述字面值的数据类型并说明每一组内几种字面值的区别：

(a) 'a', L'a', "a", L"a"
(b) 10, 10u, 10L, 10uL, 012, 0xC
(c) 3.14, 3.14f, 3.14L
(d) 10, 10u, 10., 10e-2

【出题思路】

本题考查的知识点是利用特殊的前缀和后缀指定字面值的类型。

【解答】

(a) 'a'表示字符 a，L'a'表示宽字符型字面值 a 且类型是 wchar_t，" a " 表示字符串 a，L " a " 表示宽字符型字符串 a。

(b) 10 是一个普通的整数类型字面值，10u 表示一个无符号数，10L 表示一个长整型数，10uL 表示一个无符号长整型数，012 是一个八进制数（对应的十进制数是 10），0xC 是一个十六进制数（对应的十进制数是 12）。

(c) 3.14 是一个普通的浮点类型字面值，3.14f 表示一个 float 类型的单精度浮点数，3.14L 表示一个 long double 类型的扩展精度浮点数。

(d) 10 是一个整数，10u 是一个无符号整数，10.是一个浮点数，10e-2 是一

个科学计数法表示的浮点数，大小为 $10*10^{-2}=0.1$。

**练习 2.6：**下面两组定义是否有区别，如果有，请叙述之：

```
int month = 9, day = 7;
int month = 09, day = 07;
```

【出题思路】

本题旨在考查十进制数字与八进制数字的表示方法。

【解答】

第一组定义是正确的，定义了两个十进制数 9 和 7。

第二组定义是错误的，编译时将报错。因为以 0 开头的数是八进制数，而数字 9 显然超出了八进制数能表示的范围，所以第二组定义无法被编译通过。

**练习 2.7：**下述字面值表示何种含义？它们各自的数据类型是什么？

(a) `"Who goes with F\145rgus?\012"`
(b) `3.14e1L`　　(c) `1024f`　　(d) `3.14L`

【出题思路】

本题考查的知识点是转义字符及利用特殊的后缀指定字面值的类型。

【解答】

(a)是一个字符串，包含两个转义字符，其中`\145` 表示字符 `e`，`\012` 表示一个换行符，因此该字符串的输出结果是 `Who goes with Fergus?`

(b)是一个科学计数法表示的扩展精度浮点数，大小为 $3.14*10^{1}=31.4$。

(c)试图表示一个单精度浮点数，但是该形式在某些编译器中将报错，因为后缀 `f` 直接跟在了整数 `1024` 后面；改写成 `1024.f` 就可以了。

(d)是一个扩展精度浮点数，类型是 `long double`，大小为 3.14。

**练习 2.8：**请利用转义字符编写一段程序，要求先输出 `2M`，然后转到新一行。修改程序使其先输出 `2`，然后输出制表符，再输出 `M`，最后转到新一行。

【出题思路】

本题旨在考查转义字符的使用。

【解答】

```
#include <iostream>

int main ()
{
    std::cout << "2\x4d\012";        // 输出 2M，然后换行
    std::cout << "2\tM\n";           // 输出 2、制表符、M，然后换行
    return 0;
}
```

主函数的前两行分别实现了题目中要求的两种输出形式。

其中，字符串"2\x4d\012"先输出字符 2，紧接着利用转义字符\x4d 输出字符 M，最后利用转义字符\012 转到新一行。

字符串"2\tM\n"先输出字符 2，然后利用转义字符\t 输出一个制表符，接着输出字符 M，最后利用转义字符\n 转到新一行。

读者可以发现，输出同一个字符有多种方式可供选择。例如，可以直接输出字符 M，也可以通过转义字符\x4d 输出字符 M；可以用转义字符\012 换行，也可以用转义字符\n 换行。

**练习 2.9：**解释下列定义的含义。对于非法的定义，请说明错在何处并将其改正。

```
(a) std::cin >> int input_value;      (b) int i = { 3.14 };
(c) double salary = wage = 9999.99;   (d) int i = 3.14;
```

【出题思路】

本题旨在考查变量定义与初始化，其中列表初始化是重点也是难点。

【解答】

(a)是错误的，输入运算符的右侧需要一个明确的变量名称，而非定义变量的语句，改正后的结果是：

```
int input_value;
   std::cin >> input_value;
```

(b)引发警告，该语句定义了一个整型变量 i，但是试图通过列表初始化的方式把浮点数 3.14 赋值给 i，这样做将造成小数部分丢失，是一种不被建议的窄化操作。

(c)是错误的，该语句试图将 9999.99 分别赋值给 salary 和 wage，但是在声明语句中声明多个变量时需要用逗号将变量名隔开，而不能直接用赋值运算符连接，改正后的结果是：

```
double salary, wage;
   salary = wage = 9999.99;
```

(d)引发警告，该语句定义了一个整型变量 i，但是试图把浮点数 3.14 赋值给 i，这样做将造成小数部分丢失，与(b)一样是不被建议的窄化操作。

**练习 2.10：**下列变量的初值分别是什么？

```
std::string global_str;
int global_int;
int main()
{
   int local_int;
   std::string local_str;
}
```

【出题思路】

本题旨在考查默认初始化的几种不同情况，如全局变量和局部变量的区别、内置类型和复合类型的区别。

【解答】

对于 string 类型的变量来说，因为 string 类型本身接受无参数的初始化方式，所以不论变量定义在函数内还是函数外都被默认初始化为空串。

对于内置类型 int 来说，变量 global_int 定义在所有函数体之外，根据 C++ 的规定，global_int 默认初始化为 0；而变量 local_int 定义在 main 函数的内部，将不被初始化，如果程序试图拷贝或输出未初始化的变量，将遇到一个未定义的奇异值。

**练习 2.11：**指出下面的语句是声明还是定义：

```
(a) extern int ix = 1024;
(b) int iy;
(c) extern int iz;
```

【出题思路】

本题旨在考查变量声明和定义的关系。

【解答】

声明与定义的关系是：声明使得名字为程序所知，而定义负责创建与名字关联的实体。(a)定义了变量 ix，(b)声明并定义了变量 iy，(c)声明了变量 iz。

**练习 2.12：**请指出下面的名字哪个是非法的?

```
(a) int double = 3.14;          (b) int _;
(c) int catch-22;               (d) int 1_or_2 = 1;
(e) double Double = 3.14;
```

【出题思路】

本题旨在考查 C++标识符的命名规则。

【解答】

(a)是非法的，因为 double 是 C++关键字，代表一种数据类型，不能作为变量的名字。

(c)是非法的，在标识符中只能出现字母、数字和下画线，不能出现符号-，如果改成“int catch_22;”就是合法的了。

(d)是非法的，因为标识符必须以字母或下画线开头，不能以数字开头。

(b)和(e)是合法的命名。

**练习 2.13：**下面程序中 j 的值是多少?

```
int i = 42;
int main()
{
  int i = 100;
  int j = i;
}
```

【出题思路】

本题旨在考查全局作用域与局部作用域的关系。

【解答】

j 的值是 100。C++允许内层作用域重新定义外层作用域中已有的名字，在本题中，int i=42;位于外层作用域，但是变量 i 在内层作用域被重新定义了，因此真正赋予 j 的值是定义在内层作用域中的 i 的值，即 100。

**练习 2.14：**下面的程序合法吗？如果合法，它将输出什么？

```
int i = 100, sum = 0;
for (int i = 0; i != 10; ++i)
  sum += i;
std::cout << i << " " << sum << std::endl;
```

【出题思路】

本题旨在考查嵌套作用域中变量的定义和使用。

【解答】

该程序是合法的，输出结果是 100  45。

该程序存在嵌套的作用域，其中 for 循环之外是外层作用域，for 循环内部是内层作用域。首先在外层作用域中定义了 i 和 sum，但是在 for 循环内部 i 被重新定义了，因此 for 循环实际上是从 i=0 循环到了 i=9，内层作用域中没有重新定义 sum，因此 sum 的初始值是 0 并在此基础上依次累加。最后一句输出语句位于外层作用域中，此时在 for 循环内部重新定义的 i 已经失效，因此实际输出的仍然是外层作用域的 i，值为 100；而 sum 经由循环累加，值变为了 45。

**练习 2.15：**下面的哪个定义是不合法的？为什么？

```
(a) int ival = 1.01;        (b) int &rval1 = 1.01;
(c) int &rval2 = ival;      (d) int &rval3;
```

【出题思路】

本题旨在考查引用的含义，应该明确引用与对象的关系。

【解答】

(b)是非法的，引用必须指向一个实际存在的对象而非字面值常量。

(d)是非法的，因为我们无法令引用重新绑定到另外一个对象，所以引用必须初始化。

(a)和(c)是合法的。

**练习 2.16：**考查下面的所有赋值然后回答：哪些赋值是不合法的？为什么？哪些赋值是合法的？它们执行了什么样的操作？

```
int i = 0, &r1 = i; double d = 0, &r2 = d;
(a) r2 = 3.14159;           (b) r2 = r1;
(c) i = r2;                 (d) r1 = d;
```

【出题思路】

本题旨在考查引用的含义及用法，应该明确引用与对象的关系。

【解答】

(a)是合法的，为引用赋值实际上是把值赋给了与引用绑定的对象，在这里是把 3.14159 赋给了变量 d。

(b)是合法的，以引用作为初始值实际上是以引用绑定的对象作为初始值，在这里是把 i 的值赋给了变量 d。

(c)是合法的，把 d 的值赋给了变量 i，因为 d 是双精度浮点数而 i 是整数，所以该语句实际上执行了窄化操作。

(d)是合法的，把 d 的值赋给了变量 i，与上一条语句一样执行了窄化操作。

**练习 2.17：**执行下面的代码段将输出什么结果?

```
int i, &ri = i;
i = 5; ri = 10;
std::cout << i << " " << ri << std::endl;
```

【出题思路】

本题旨在考查引用的含义及用法，应该明确引用与对象的关系。

【解答】

程序的输出结果是 10　10。

引用不是对象，它只是为已经存在的对象起了另外一个名字，因此 ri 实际上是 i 的别名。在上述程序中，首先将 i 赋值为 5，然后把这个值更新为 10。因为 ri 是 i 的引用，所以它们的输出结果是一样的。

**练习 2.18：**编写代码分别更改指针的值以及指针所指对象的值。

【出题思路】

本题旨在考查指针的含义，以及指针本身的值与指针所指对象值的区别，重点掌握解引用运算符的使用方法。

【解答】

一个满足要求的简单示例如下所示：

```
#include <iostream>

int main ()
{
    int i = 5, j = 10;
    int *p = &i;
    std::cout << p << " " << *p << std::endl;
    p=&j;
    std::cout << p << " " << *p << std::endl;
    *p=20;
    std::cout << p << " " << *p << std::endl;
    j=30;
    std::cout << p << " " << *p << std::endl;
    return 0;
}
```

程序的输出结果是：

```
0x28fef8  5
0x28fef4  10
0x28fef4  20
0x28fef4  30
```

在上述示例中，首先定义了两个整型变量 i 和 j 以及一个整型指针 p，初始情况下令指针 p 指向变量 i，此时分别输出 p 的值（即 p 所指对象的内存地址）以及 p 所指对象的值，得到 0x28fef8 和 5。

随后依次更改指针的值以及指针所指对象的值。p=&j;更改了指针的值，令指针 p 指向另外一个整数对象 j。*p=20;和 j=30 是两种更改指针所指对象值的方式，前者显式地更改指针 p 所指的内容，后者则通过更改变量 j 的值实现同样的目的。

**练习 2.19：**说明指针和引用的主要区别。

【出题思路】

指针和引用同为复合类型，都与内存中实际存在的对象有联系。本题旨在考查二者的主要区别。

【解答】

指针“指向”内存中的某个对象，而引用“绑定到”内存中的某个对象，它们都实现了对其他对象的间接访问，二者的区别主要有两方面：

第一，指针本身就是一个对象，允许对指针赋值和拷贝，而且在指针的生命周期内它可以指向几个不同的对象；引用不是一个对象，无法令引用重新绑定到另外一个对象。

第二，指针无须在定义时赋初值，和其他内置类型一样，在块作用域内定义的指针如果没有被初始化，也将拥有一个不确定的值；引用则必须在定义时赋初值。

**练习 2.20：**请叙述下面这段代码的作用。

```
int i = 42;
int *p1 = &i;
*p1 = *p1 * *p1;
```

【出题思路】

本题旨在考查运算符*的两种含义：一是指针声明符，二是解引用运算符。

【解答】

这段代码首先定义了一个整型变量 i 并设其初值为 42；接着定义了一个整型指针 p1，令其指向变量 i；最后取出 p1 所指的当前值，计算平方后重新赋给 p1 所指的变量 i。

第二行的*表示声明一个指针，第三行的*表示解引用运算，即取出指针 p1 所指对象的值。

**练习 2.21：**请解释下述定义。在这些定义中有非法的吗？如果有，为什么？

```
int i = 0;
(a) double* dp = &i; (b) int *ip = i; (c) int *p = &i;
```

【出题思路】

本题旨在考查指针的声明和定义。

【解答】

(a)是非法的，dp 是一个 double 指针，而 i 是一个 int 变量，类型不匹配。

(b)是非法的，不能直接把 int 变量赋给 int 指针，正确的做法是通过取地址运算&i 得到变量 i 在内存中的地址，然后再将该地址赋给指针。

(c)是合法的。

**练习 2.22：**假设 p 是一个 int 型指针，请说明下述代码的含义。

```
if (p)    // ...
if (*p)   // ...
```

【出题思路】

本题旨在考查指针的值与指针所指对象的值的区别，巧妙之处是把 p 和*p 作为 if 语句的条件，通过判断其是否为真帮助读者加深理解。

【解答】

指针 p 作为 if 语句的条件时，实际检验的是指针本身的值，即指针所指的地址值。如果指针指向一个真实存在的变量，则其值必不为 0，此时条件为真；如果指针没有指向任何对象或者是无效指针，则对 p 的使用将引发不可预计的结果。

解引用运算符*p 作为 if 语句的条件时，实际检验的是指针所指的对象内容，在上面的示例中是指针 p 所指的 int 值。如果该 int 值为 0，则条件为假；否则，如果该 int 值不为 0，对应条件为真。

一个简单的示例如下所示：

```
#include <iostream>

int main ()
{
    int i = 0;
    int *p1 = nullptr;
    int *p = &i;
    if (p1)  // 检验指针的值（即指针所指对象的地址）
        std::cout << "p1 pass" << std::endl;
    if (p)   // 检验指针的值（即指针所指对象的地址）
        std::cout << "p pass" << std::endl;
    if (*p)  // 检验指针所指对象的值
        std::cout << "i pass" << std::endl;
    return 0;
}
```

在这段程序中，p 和 p1 是两个整型指针，其中 p1 被定义为空指针(nullptr)，p 则指向整数 i。在 3 个判断条件中，p1 指向为空，意即指向地址为 0，条件不满

足；p 指向 i，在内存中有一个实际的地址值且不为 0，因此该条件满足；*p 表示 p 所指对象的内容，即整数 i 的值，因为程序开始处 i 被赋予了初值 0，所以这个条件不满足。

综合以上分析，程序的输出结果是 p  pass。

**练习 2.23：**给定指针 p，你能知道它是否指向了一个合法的对象吗？如果能，叙述判断的思路；如果不能，说明原因。

【出题思路】

本题旨在考查指针初始化，读者应该熟悉并掌握 C++11 的新语法特征 nullptr。

【解答】

在 C++程序中，应该尽量初始化所有指针，并且尽可能等定义了对象之后再定义指向它的指针。如果实在不清楚指针应该指向何处，就把它初始化为 nullptr 或者 0，这样程序就能检测并知道它有没有指向一个具体的对象了。其中，nullptr 是 C++11 新标准刚刚引入的一个特殊字面值，它可以转换成任意其他的指针类型。在此前提下，判断 p 是否指向合法的对象，只需把 p 作为 if 语句的条件即可，如果 p 的值是 nullptr，则条件为假；反之，条件为真。

如果不注意初始化所有指针而贸然判断指针的值，则有可能引发不可预知的结果。一种处理的办法是把 if(p) 置于 try 结构中，当程序块顺利执行时，表示 p 指向了合法的对象；当程序块出错跳转到 catch 语句时，表示 p 没有指向合法的对象。

**练习 2.24：**在下面这段代码中为什么 p 合法而 lp 非法？

```
int i = 42; void *p = &i; long *lp = &i;
```

【出题思路】

本题主要考查 void*指针的含义和用法。

【解答】

p 是合法的，因为 void*是一种特殊的指针类型，可用于存放任意对象的地址。

lp 是非法的，因为 lp 是一个长整型指针，而 i 只是一个普通整型数，二者的类型不匹配。

**练习 2.25：**说明下列变量的类型和值。

```
(a) int* ip, i, &r = i; (b) int i, *ip = 0; (c) int* ip, ip2;
```

【出题思路】

本题旨在考查指针和引用这两种复合类型的声明。

【解答】

(a)ip 是一个整型指针，指向一个整型数，它的值是所指整型数在内存中的地址；i 是一个整型数；r 是一个引用，它绑定了 i，可以看作是 i 的别名，r 的值就是 i 的值。

(b)i 是一个整型数；ip 是一个整型指针，但是它不指向任何具体的对象，它的值被初始化为 0。

(c)ip 是一个整型指针，指向一个整型数，它的值是所指整型数在内存中的地址；ip2 是一个整型数。

**练习 2.26：**下面哪些句子是合法的？如果有不合法的句子，请说明为什么？

```
(a) const int buf;              (b) int cnt = 0;
(c) const int sz = cnt;         (d) ++cnt; ++sz;
```

【出题思路】

本题旨在考查 const 限定符的用法，尤其是 const 对象的定义、初始化和运算。

【解答】

本题的所有语句应该被看作是顺序执行的，即形如：

```
const int buf;
int cnt = 0;
const int sz = cnt;
++cnt;
++sz;
```

(a)是非法的，const 对象一旦创建后其值就不能改变，所以 const 对象必须初始化。该句应修改为 const int buf = 10。

(b)和(c)是合法的。

(d)是非法的，sz 是一个 const 对象，其值不能被改变，当然不能执行自增操作。

**练习 2.27：**下面的哪些初始化是合法的？请说明原因。

```
(a) int i = -1, &r = 0;         (b) int *const p2 = &i2;
(c) const int i = -1, &r = 0;(d) const int *const p3 = &i2;
(e) const int *p1 = &i2;        (f) const int &const r2;
(g) const int i2 = i, &r = i;
```

【出题思路】

本题旨在考查常量引用、常量指针和指向常量的指针的初始化方法。

【解答】

(a)是非法的，非常量引用 r 不能引用字面值常量 0。

(b)是合法的，p2 是一个常量指针，p2 的值永不改变，即 p2 永远指向变量 i2。

(c)是合法的，i 是一个常量，r 是一个常量引用，此时 r 可以绑定到字面值常量 0。

(d)是合法的，p3 是一个常量指针，p3 的值永不改变，即 p3 永远指向变量 i2；同时 p3 指向的是常量，即我们不能通过 p3 改变所指对象的值。

(e)是合法的，p1 指向一个常量，即我们不能通过 p1 改变所指对象的值。

(f)是非法的，引用本身不是对象，因此不能让引用恒定不变。

(g)是合法的，i2 是一个常量，r 是一个常量引用。

**练习 2.28：**说明下面的这些定义是什么意思，挑出其中不合法的。

```
(a) int i, *const cp;          (b) int *p1, *const p2;
(c) const int ic, &r = ic;     (d) const int *const p3;
(e) const int *p;
```

【出题思路】

本题旨在考查常量引用、常量指针和指向常量的指针的定义及区别。

【解答】

(a)是非法的，cp 是一个常量指针，因其值不能被改变，所以必须初始化。

(b)是非法的，cp2 是一个常量指针，因其值不能被改变，所以必须初始化。

(c)是非法的，ic 是一个常量，因其值不能被改变，所以必须初始化。

(d)是非法的，p3 是一个常量指针，因其值不能被改变，所以必须初始化；同时 p3 指向的是常量，即我们不能通过 p3 改变所指对象的值。

(e)是合法的，但是 p 没有指向任何实际的对象。

**练习 2.29：**假设已有上一个练习中定义的那些变量，则下面的哪些语句是合法的？请说明原因。

```
(a) i = ic;                (b) p1 = p3;
(c) p1 = &ic;              (d) p3 = &ic;
(e) p2 = p1;               (f) ic = *p3;
```

【出题思路】

本题旨在考查常量引用、常量指针和指向常量的指针的赋值方法。

【解答】

(a)是合法的，常量 ic 的值赋给了非常量 i。

(b)是非法的，普通指针 p1 指向了一个常量，从语法上说，p1 的值可以随意改变，显然是不合理的。

(c)是非法的，普通指针 p1 指向了一个常量，错误情况与上一条类似。

(d)是非法的，p3 是一个常量指针，不能被赋值。

(e)是非法的，p2 是一个常量指针，不能被赋值。

(f)是非法的，ic 是一个常量，不能被赋值。

**练习 2.30：**对于下面的这些语句，请说明对象被声明成了顶层 const 还是底层 const？

```
const int v2 = 0;   int v1 = v2;
int *p1 = &v1, &r1 = v1;
const int *p2 = &v2, *const p3 = &i, &r2 = v2;
```

【出题思路】

本题旨在考查顶层 const 和底层 const 的区别，读者应明确顶层 const 表示任意的对象是常量，而底层 const 与指针和引用等复合类型的基本类型部分有关。

【解答】

v2 和 p3 是顶层 const，分别表示一个整型常量和一个整型常量指针；p2 和

r2 是底层 const，分别表示它们所指（所引用）的对象是常量。

**练习 2.31：**假设已有上一个练习中所做的那些声明，则下面的哪些语句是合法的？请说明顶层 const 和底层 const 在每个例子中有何体现。

```
r1 = v2;
p1 = p2; p2 = p1;
p1 = p3; p2 = p3;
```

【出题思路】

本题旨在考查顶层 const 和底层 const 对于拷贝操作的影响。

【解答】

在执行拷贝操作时，顶层 const 和底层 const 区别明显。其中，顶层 const 不受影响，这是因为拷贝操作并不会改变被拷贝对象的值。底层 const 的限制则不容忽视，拷入和拷出的对象必须具有相同的底层 const 资格，或者两个对象的数据类型必须能够转换。一般来说，非常量可以转换成常量，反之则不行。

r1=v2;是合法的，r1 是一个非常量引用，v2 是一个常量（顶层 const），把 v2 的值拷贝给 r1 不会对 v2 有任何影响。

p1=p2;是非法的，p1 是普通指针，指向的对象可以是任意值，p2 是指向常量的指针（底层 const），令 p1 指向 p2 所指的内容，有可能错误地改变常量的值。

p2=p1;是合法的，与上一条语句相反，p2 可以指向一个非常量，只不过我们不会通过 p2 更改它所指的值。

p1=p3;是非法的，p3 包含底层 const 定义（p3 所指的对象是常量），不能把 p3 的值赋给普通指针。

p2=p3;是合法的，p2 和 p3 包含相同的底层 const，p3 的顶层 const 则可以忽略不计。

**练习 2.32：**下面的代码是否合法？如果非法，请设法将其修改正确。

```
int null = 0, *p = null;
```

【出题思路】

本题旨在考查指针的使用。

【解答】

上述代码是非法的，null 是一个 int 变量，p 是一个 int 指针，二者不能直接绑定。仅从语法角度来说，可以将代码修改为：

```
int null = 0, *p = &null;
```

显然，这种改法与代码的原意不一定相符。另一种改法是使用 nullptr：

```
int null = 0, *p = nullptr;
```

**练习 2.33：**利用本节定义的变量，判断下列语句的运行结果。

```
a = 42; b = 42; c = 42;
```

```
    d = 42; e = 42; g = 42;
```

【出题思路】

本题旨在考查 auto 说明符与复合类型、常量混合使用时的各种情形。首先，使用引用其实是使用引用的对象，所以当引用被用作初始值时，真正参与初始化的其实是引用对象的值，编译器以引用对象的类型作为 auto 的推断类型。其次，auto 一般会忽略掉顶层 const，同时保留底层 const。

【解答】

前 3 条赋值语句是合法的，原因如下：

r 是 i 的别名，而 i 是一个整数，所以 a 的类型推断结果是一个整数；ci 是一个整型常量，在类型推断时顶层 const 被忽略掉了，所以 b 是一个整数；cr 是 ci 的别名，而 ci 是一个整型常量，所以 c 的类型推断结果是一个整数。因为 a、b、c 都是整数，所以为其赋值 42 是合法的。

后 3 条赋值语句是非法的，原因如下：

i 是一个整数，&i 是 i 的地址，所以 d 的类型推断结果是一个整型指针；ci 是一个整型常量，&ci 是一个整型常量的地址，所以 e 的类型推断结果是一个指向整型常量的指针；ci 是一个整型常量，所以 g 的类型推断结果是一个整型常量引用。因为 d 和 e 都是指针，所以不能直接用字面值常量为其赋值；g 绑定到了整型常量，所以不能修改它的值。

**练习 2.34：**基于上一个练习中的变量和语句编写一段程序，输出赋值前后变量的内容，你刚才的推断正确吗？如果不对，请反复研读本节的示例直到你明白错在何处为止。

【出题思路】

本题旨在考查 auto 说明符与复合类型、常量混合使用时的各种情形。

【解答】

基于上一个练习中的变量和语句编写的程序如下所示：

```
#include <iostream>

int main()
{
    int i = 0, &r = i;
    auto a = r;       // a 是一个整数（r 是 i 的别名，而 i 是个整数）
    const int ci = i, &cr = ci;
    auto b = ci;      // b 是一个整数（ci 的顶层 const 特性被忽略掉了）
    auto c = cr;      // c 是一个整数（cr 是 ci 的别名，ci 本身是一个顶层 const）
    auto d = &i;      // d 是一个整型指针（整数的地址就是指向整数的指针）
    auto e = &ci;  // e 是一个指向整型常量的指针（对常量对象取地址是一种底层 const）
    auto &g = ci;     // g 是一个整型常量引用，绑定到 ci
    std::cout << a << " " << b << " " << c << " " << d << " " << e <<
" " << g << std::endl;
```

```
    a = 42;
    b = 42;
    c = 42;
    d = 42;          // 错误：d 是一个指针，赋值非法
    e = 42;          // 错误：e 是一个指针，赋值非法
    g = 42;          // 错误：g 是一个常量引用，赋值非法
    std::cout << a << " " << b << " " << c << " " << d << " " << e <<
" " << g << std::endl;

    return 0;
}
```

程序的输出结果是：

```
0    0   0   0x28fedc    0x28fed8    0
42  42  42   0x28fedc    0x28fed8    0
```

**练习 2.35：**判断下列定义推断出的类型是什么，然后编写程序进行验证。

```
const int i = 42;
auto j = i; const auto &k = i; auto *p = &i;
const auto j2 = i, &k2 = i;
```

【出题思路】

本题旨在考查 auto 说明符与复合类型、常量混合使用时的各种情形。

【解答】

由题意可知，i 是一个整型常量，j 的类型推断结果是整数，k 的类型推断结果是整型常量，p 的类型推断结果是指向整型常量的指针，j2 的类型推断结果是整数，k2 的类型推断结果是整数。

用于验证的程序是：

```
#include <iostream>
#include<typeinfo>

int main ()
{
    const int i = 42;
    auto j = i;
    const auto &k = i;
    auto *p = &i;
    const auto j2 = i, &k2 = i;
    std::cout << typeid(i).name() << std::endl;
    std::cout << typeid(j).name() << std::endl;
    std::cout << typeid(k).name() << std::endl;
    std::cout << typeid(p).name() << std::endl;
    std::cout << typeid(j2).name() << std::endl;
    std::cout << typeid(k2).name() << std::endl;

    return 0;
}
```

**练习 2.36：**关于下面的代码，请指出每一个变量的类型以及程序结束时它们各自的值。

```
int a = 3, b = 4;
decltype(a) c = a;
decltype((b)) d = a;
++c;
++d;
```

【出题思路】

本题旨在考查 decltype 与引用的关系。对于 decltype 所用的表达式来说，如果变量名加上一对括号，得到的类型与不加括号时会有不同。具体来说，如果 decltype 使用的是一个不加括号的变量，则得到的结果就是该变量的类型；如果给变量加上了一层或多层括号，编译器就会把它当成是一个表达式，从而推断得到引用类型。

【解答】

在本题的程序中，初始情况下 a 的值是 3、b 的值是 4。decltype(a)　c=a;使用的是一个不加括号的变量，因此 c 的类型就是 a 的类型，即该语句等同于 int c=a;，此时 c 是一个新整型变量，值为 3。decltype((b))　d=a;使用的是一个加了括号的变量，因此 d 的类型是引用，即该语句等同于 int　&d=a;，此时 d 是变量 a 的别名。

执行++c;　++d;时，变量 c 的值自增为 4，因为 d 是 a 的别名，所以 d 自增 1 意味着 a 的值变成了 4。当程序结束时，a、b、c、d 的值都是 4。

**练习 2.37：**赋值是会产生引用的一类典型表达式，引用的类型就是左值的类型。也就是说，如果 i 是 int，则表达式 i=x 的类型是 int&。根据这一特点，请指出下面的代码中每一个变量的类型和值。

```
int a = 3, b = 4;
decltype(a) c = a;
decltype(a = b) d = a;
```

【出题思路】

decltype 的参数既可以是普通变量，也可以是一个表达式。当参数是普通变量时，推断出的类型就是该变量的类型；当参数是表达式时，推断出的类型是引用。

【解答】

根据 decltype 的上述性质可知，c 的类型是 int，值为 3；表达式 a=b 作为 decltype 的参数，编译器分析表达式并得到它的类型作为 d 的推断类型，但是不实际计算该表达式，所以 a 的值不发生改变，仍然是 3；d 的类型是 int&，d 是 a 的别名，值是 3；b 的值一直没有发生改变，为 4。

**练习 2.38：**说明由 decltype 指定类型和由 auto 指定类型有何区别。请举出一个例子，decltype 指定的类型与 auto 指定的类型一样；再举一个例子，decltype 指定的类型与 auto 指定的类型不一样。

【出题思路】

auto 和 decltype 是两种类型推断的方式，本题旨在考查二者的区别和联系。

【解答】

auto 和 decltype 的区别主要有三个方面：

第一，auto 类型说明符用编译器计算变量的初始值来推断其类型，而 decltype 虽然也让编译器分析表达式并得到它的类型，但是不实际计算表达式的值。

第二，编译器推断出来的 auto 类型有时候和初始值的类型并不完全一样，编译器会适当地改变结果类型使其更符合初始化规则。例如，auto 一般会忽略掉顶层 const，而把底层 const 保留下来。与之相反，decltype 会保留变量的顶层 const。

第三，与 auto 不同，decltype 的结果类型与表达式形式密切相关，如果变量名加上了一对括号，则得到的类型与不加括号时会有不同。如果 decltype 使用的是一个不加括号的变量，则得到的结果就是该变量的类型；如果给变量加上了一层或多层括号，则编译器将推断得到引用类型。

一个用以说明的示例如下所示：

```
#include <iostream>
#include <typeinfo>

int main ()
{
    int a = 3;
    auto c1 = a;
    decltype(a) c2 = a;
    decltype((a)) c3 = a;

    const int d = 5;
    auto f1 = d;
    decltype(d) f2 = d;

    std::cout << typeid(c1).name() << std::endl;
    std::cout << typeid(c2).name() << std::endl;
    std::cout << typeid(c3).name() << std::endl;
    std::cout << typeid(f1).name() << std::endl;
    std::cout << typeid(f2).name() << std::endl;

    c1++;
    c2++;
    c3++;
    f1++;
    f2++;        // 错误：f2 是整型常量，不能执行自增操作
    std::cout << a << " " << c1 << " " << c2 << " " << c3 << " " << f1
<< " " << f2 << std::endl;
    return 0;
}
```

对于第一组类型推断来说，a 是一个非常量整数，c1 的推断结果是整数，c2 的推断结果也是整数，c3 的推断结果由于变量 a 额外加了一对括号所以是整数引用。c1、c2、c3 依次执行自增操作，因为 c3 是变量 a 的别名，所以 c3 自增等同于 a

自增，最终 a、c1、c2、c3 的值都变为 4。

对于第二组类型推断来说，d 是一个常量整数，含有顶层 const，使用 auto 推断类型自动忽略掉顶层 const，因此 f1 的推断结果是整数；decltype 则保留顶层 const，所以 f2 的推断结果是整数常量。f1 可以正常执行自增操作，而常量 f2 的值不能被改变，所以无法自增。

**练习 2.39：**编译下面的程序观察其运行结果，注意，如果忘记写类定义体后面的分号会发生什么情况？记录下相关信息，以后可能会有用。

```
struct Foo { /* 此处为空 */ }      // 注意：没有分号
int main()
{
  return 0;
}
```

【出题思路】

本题旨在考查类定义的语法规范，尤其要注意类体结束之后的分号必不可少。

【解答】

该程序无法编译通过，原因是缺少了一个分号。因为类体后面可以紧跟变量名以示对该类型对象的定义，所以在类体右侧表示结束的花括号之后必须写一个分号。稍作修改，该程序就可以编译通过了。

```
struct Foo { /* 此处为空 */ };
int main ()
{
   return 0;
}
```

**练习 2.40：**根据自己的理解写出 Sales_data 类，最好与书中的例子有所区别。

【出题思路】

类的设计源于实际应用，设计 Sales_data 类的关键是理解在销售过程中应该包含哪些数据元素，同时为每个元素设定合理的数据类型。

【解答】

原书中的程序包含 3 个数据成员，分别是 bookNo（书籍编号）、units_sold（销售量）、revenue（销售收入），新设计的 Sales_data 类细化了销售收入的计算方式，在保留 bookNo 和 units_sold 的基础上，新增了 sellingprice（零售价、原价）、saleprice（实售价、折扣价）、discount（折扣），其中 discount=saleprice/sellingprice。

```
struct Sales_data {
    std::string bookNo;              // 书籍编号
    unsigned units_sold = 0;         // 销售量
    double sellingprice = 0.0;       // 零售价
    double saleprice = 0.0;          // 实售价
```

```
    double discount = 0.0          // 折扣

};
```

**练习 2.41：**使用你自己的 Sales_data 类重写 1.5.1 节（第 20 页）、1.5.2 节（第 21 页）和 1.6 节（第 22 页）的练习。眼下先把 Sales_data 类的定义和 main 函数放在同一个文件里。

【出题思路】

本题旨在考查如何利用自定义类创建对象并执行基本操作。

【解答】

```
#include <iostream>
#include <string>

using namespace std;

class Sales_data {
// 友元函数
friend std::istream& operator >> (std::istream&, Sales_data&);
// 友元函数
friend std::ostream& operator << (std::ostream&, const Sales_data&);
// 友元函数
friend bool operator < (const Sales_data&, const Sales_data&);
// 友元函数
friend bool operator == (const Sales_data&, const Sales_data&);
public:     // 构造函数的 3 种形式
   Sales_data() = default;
   Sales_data(const std::string &book): bookNo(book) { }
   Sales_data(std::istream &is) { is >> *this; }
public:
   Sales_data& operator += (const Sales_data&);
   std::string isbn() const { return bookNo; }
private:
   std::string bookNo;          // 书籍编号，隐式初始化为空串
   unsigned units_sold = 0;     // 销售量，显式初始化为 0
   double sellingprice = 0.0;   // 原始价格，显式初始化为 0.0
   double saleprice = 0.0;      // 实售价格，显式初始化为 0.0
   double discount = 0.0;       // 折扣，显式初始化为 0.0

};

inline bool compareIsbn(const Sales_data &lhs, const Sales_data &rhs)
{ return lhs.isbn() == rhs.isbn(); }

Sales_data operator + (const Sales_data&, const Sales_data&);

inline bool operator == (const Sales_data &lhs, const Sales_data &rhs)
{
   return lhs.units_sold == rhs.units_sold &&
         lhs.sellingprice == rhs.sellingprice &&
```

```
            lhs.saleprice == rhs.saleprice &&
            lhs.isbn() == rhs.isbn();
}

inline bool operator != (const Sales_data &lhs, const Sales_data &rhs)
{
    return !(lhs == rhs);    // 基于运算符==给出!=的定义
}

Sales_data& Sales_data::operator += (const Sales_data& rhs)
{
    units_sold += rhs.units_sold;
    saleprice = (rhs.saleprice * rhs.units_sold + saleprice * units_sold)
       / (rhs.units_sold + units_sold);
    if(sellingprice != 0)
        discount = saleprice / sellingprice;
    return *this;
}

Sales_data operator + (const Sales_data& lhs, const Sales_data& rhs)
{
    Sales_data ret(lhs); // 把 lhs 的内容拷贝到临时变量 ret 中，这种做法便于运算
    ret += rhs;          // 把 rhs 的内容加入其中
    return ret;          // 返回 ret
}

std::istream& operator>>(std::istream& in, Sales_data& s)
{
    in >> s.bookNo >> s.units_sold >> s.sellingprice >> s.saleprice;
    if (in && s.sellingprice != 0)
        s.discount = s.saleprice / s.sellingprice;
    else
        s = Sales_data();    // 输入错误，重置输入的数据
    return in;
}

std::ostream& operator << (std::ostream& out, const Sales_data& s)
{
    out << s.isbn() << " " << s.units_sold << " "
        << s.sellingprice << " " << s.saleprice << " " << s.discount;
    return out;
}

int main()
{
    Sales_data book;
    std::cout << "请输入销售记录："<< std::endl;
    while (std::cin >> book) {
        std::cout << " ISBN、售出本数、原始价格、实售价格、折扣为" << book <<
std::endl;
    }
    Sales_data trans1, trans2;
```

```
    std::cout << "请输入两条 ISBN 相同的销售记录: "<< std::endl;
    std::cin >> trans1 >> trans2;
    if (compareIsbn(trans1, trans2))
       std::cout << "汇总信息: ISBN、售出本数、原始价格、实售价格、折扣为 "
       << trans1 + trans2 << std::endl;
    else
       std::cout << "两条销售记录的 ISBN 不同" << std::endl;

    Sales_data total, trans;
    std::cout << "请输入几条 ISBN 相同的销售记录: "<< std::endl;
    if (std::cin >> total) {
       while (std::cin >> trans)
          if (compareIsbn(total, trans)) // ISBN 相同
             total = total + trans;
          else { // ISBN 不同
             std::cout << "当前书籍 ISBN 不同" << std::endl;
             break;
          }
       std::cout << "有效汇总信息: ISBN、售出本数、原始价格、实售价格、折扣为"
          << total << std::endl;
    }
    else {
       std::cout << "没有数据" << std::endl;
       return -1;
    }

    int num = 1;      // 记录当前书籍的销售记录总数
    std::cout << "请输入若干销售记录: "<< std::endl;
    if (std::cin >> trans1) {
       while (std::cin >> trans2)
          if (compareIsbn(trans1, trans2)) // ISBN 相同
             num++;
          else {   // ISBN 不同
             std::cout << trans1.isbn() << "共有"
             << num << "条销售记录" << std::endl;
             trans1 = trans2;
             num = 1;
          }
       std::cout << trans1.isbn() << "共有"
          << num << "条销售记录" << std::endl;
    }
    else {
       std::cout << "没有数据" << std::endl;
       return -1;
    }

    return 0;
}
```

**练习 2.42：**根据你自己的理解重写一个 Sales_data.h 头文件，并以此为基础重做 2.6.2 节（第 67 页）的练习。

【出题思路】

本题旨在考查自定义头文件并基于头文件编写程序的方法。

【解答】

将上题答案中类的定义放置于 Sales_data.h 头文件中，具体的使用方法与第 1 章练习 1.20～练习 1.25 类似。

Sales_data.h 头文件的内容是：

```
#ifndef SALES_DATA_H_INCLUDED
#define SALES_DATA_H_INCLUDED

#include <iostream>
#include <string>

class Sales_data {
// 友元函数
friend std::istream& operator >> (std::istream&, Sales_data&);
// 友元函数
friend std::ostream& operator << (std::ostream&, const Sales_data&);
// 友元函数
friend bool operator < (const Sales_data&, const Sales_data&);
// 友元函数
friend bool operator == (const Sales_data&, const Sales_data&);
public:     // 构造函数的 3 种形式
   Sales_data() = default;
   Sales_data(const std::string &book): bookNo(book) { }
   Sales_data(std::istream &is) { is >> *this; }
public:
   Sales_data& operator += (const Sales_data&);
   std::string isbn() const { return bookNo; }
private:
   std::string bookNo;           // 书籍编号，隐式初始化为空串
   unsigned units_sold = 0;      // 销售量，显式初始化为 0
   double sellingprice = 0.0;    // 原始价格，显式初始化为 0.0
    double saleprice = 0.0;      // 实售价格，显式初始化为 0.0
    double discount = 0.0;       // 折扣，显式初始化为 0.0

};

inline bool compareIsbn(const Sales_data &lhs, const Sales_data &rhs)
{ return lhs.isbn() == rhs.isbn(); }

Sales_data operator + (const Sales_data&, const Sales_data&);

inline bool operator == (const Sales_data &lhs, const Sales_data &rhs)
{
   return lhs.units_sold == rhs.units_sold &&
          lhs.sellingprice == rhs.sellingprice &&
          lhs.saleprice == rhs.saleprice &&
```

```
        lhs.isbn() == rhs.isbn();
}

inline bool operator != (const Sales_data &lhs, const Sales_data &rhs)
{
   return !(lhs == rhs);     // 基于运算符==给出!=的定义
}

Sales_data& Sales_data::operator += (const Sales_data& rhs)
{
   units_sold += rhs.units_sold;
   saleprice = (rhs.saleprice * rhs.units_sold + saleprice * units_sold)
      / (rhs.units_sold + units_sold);
   if(sellingprice != 0)
      discount = saleprice / sellingprice;
   return *this;
}
Sales_data operator + (const Sales_data& lhs, const Sales_data& rhs)
{
   Sales_data ret(lhs);// 把 lhs 的内容拷贝到临时变量 ret 中，这种做法便于运算
   ret += rhs;         // 把 rhs 的内容加入其中
   return ret;         // 返回 ret
}

std::istream& operator>>(std::istream& in, Sales_data& s)
{
   in >> s.bookNo >> s.units_sold >> s.sellingprice >> s.saleprice;
   if (in && s.sellingprice != 0)
      s.discount = s.saleprice / s.sellingprice;
   else
      s = Sales_data();      // 输入错误，重置输入的数据
   return in;
}

std::ostream& operator << (std::ostream& out, const Sales_data& s)
{
   out << s.isbn() << " " << s.units_sold << " "
      << s.sellingprice << " " << s.saleprice << " " << s.discount;
   return out;
}

#endif // SALES_DATA_H_INCLUDED
```

main.cpp 源文件内容是：

```
#include <iostream>
#include "Sales_data.h"

int main()
{
   Sales_data book;
   std::cout << "请输入销售记录："<< std::endl;
   while (std::cin >> book) {
      std::cout << " ISBN、售出本数、原始价格、实售价格、折扣为" << book <<
```

非常抱歉，打断了您的学习，也很抱歉上传的这本书并不完整，因为现在不劳而获的人太多，希望您能理解，在众多文件中您很幸运的下载了这本非常清晰的图书，既然您已经把这本书读到这个位置了，那么诚挚的邀请您加入 QQ 群：187541493，群内免费分享编程视频，高清 PDF 电子书，群内所分享的电子书均为重新整理过，目录详细，字体清晰，绝无歪斜。除部分经典书籍，收集整理的都是近几年的热销书籍。绝无陈旧书籍，滥竽充数。您也可以访问:www.wzbook.org 来获取完整免费的编程图书。祝您学业有成！